CS4094 MS3 Progress Report: Distributed DAG Task Scheduler

Specification: specification.md. Test-to-property table: test-matrix.md. Run logs: results/ms3-logs/. The MS2 report is kept unchanged as ms2-progress-report.md.

A. Status of every claim

Claim (spec §)	MS2	MS3	Evidence
S1 No completion without a reported success (§3)	Held	**Holds**, now including late successes from expired attempts	`LeaseTest`, lost-heartbeat, pause, partition, restart tests; `check_history` on every test
S2 No task before its dependencies (§3)	Held	**Holds**	ordering tests; `check_history` joins every worker start to its acknowledgment
S3 No duplicate completion (§3)	Held	**Holds** under crash, pause, loss, duplication and partition	duplicate/lost-reply tests, chaos run, 8-thread JUnit races
L1 Ready tasks make progress (§4)	Partial	**Holds**, including slow tasks longer than the lease	slow-task and short-partition tests, all workload tests
L2 A failed worker's task is reassigned (§4)	**Intentionally broken**	**Holds**	MS2's crash oracle now passes; crash at three attempt stages, restart, pause, partition, lost heartbeats
Fault model (§5)	Crash only, safe but stuck	Crash, restart, pause, message loss/delay/duplication, partitions all tested	14 fault tests
Performance (§6)	3 of 4 metrics	**All 4**, including recovery time, with and without failures	`make bench-recovery` (§E)

The submitted run is `results/ms3-logs/make-test-3.log` (clean source `966965e`): `mvn -B verify` passed all 77 JUnit tests, then pytest reported **102 passed, 0 failed, 0 expected failures** (exit 0). Its JUnit XML is `results/ms3-logs/junit-make-test-3.xml`, and the per-test evidence (52 directories) is in `results/ms3-logs/evidence-make-test-3/`. Re-checking all 52 of its job histories offline found no violations.

Two earlier full runs are kept as logs. `make-test-1` failed one test, which still expected MS2's exact list of metrics counters. `make-test-2` passed all 102 tests, but on code from before the idle-polling fix (§B, defect 2), and it is stamped `+uncommitted` because its own log file was being written inside the repository.

B. What changed since MS2

Leases (scheduler). Every attempt is now owned through a lease on the scheduler's monotonic clock (`LEASE_TIMEOUT_MS`, default 5 s). Assignment grants it; the start acknowledgment and a new endpoint,

`POST /v1/attempts/heartbeat`, renew it. When the deadline passes, the attempt becomes `EXPIRED`, its owner is released, a `lease_expired` event is recorded, and the task rejoins the end of the ready queue. Expiry is applied at the start of every claim, start, heartbeat and report, and by a background scan every 100 ms. Every later message from an expired attempt, including a replayed start, gets `409 LEASE_EXPIRED` and changes nothing. Under 100 lines changed in `SchedulerCore`; the happy path is unchanged.

Heartbeats (worker). While a worker holds an attempt, a background thread renews the lease every `lease/5` (read from `/v1/health`). A `409` on renewal cancels the running operation, and the worker drops the attempt without uploading or reporting. Workers also gained test-only stall points before start, during the operation and before the report.

Test harness. A per-worker fault proxy (`tests/harness/netfault.py`) can drop requests or replies, delay, duplicate, or partition one worker's link and logs every action; the harness can pause and resume workers (SIGSTOP/SIGCONT, `docker pause` on Compose). The history checker now also rejects expiry of a non-owner and any success from an expired attempt.

Tests. 17 new JUnit tests (16 deterministic lease tests on an injected clock, no sleeps, and one idle-polling memory test), 13 new fault-injection tests, and 2 new checker-falsification tests. MS2's intentional failing oracle, `test_worker_crash_reassignment`, now passes and additionally checks the recovery path and its time bound. `make test` now requires zero expected failures.

Benchmark. `benchmarks/recovery.py` (`make bench-recovery`) measures recovery time by killing busy workers mid-run, against a fault-free baseline of the same code.

Docs. The specification's claims are sharpened (§9 of the spec), the API and evidence docs cover leases, and the test-to-property matrix is new.

Defects found and fixed during MS3.

1. *A replayed start for an expired attempt was acknowledged.* It returned its cached acknowledgment. That was safe, since the worker's heartbeat and report would then be rejected, but it broke the rule that every message from an expired attempt is refused. The restart test caught it, and a JUnit assertion now covers it.
2. *Idle polling grew the scheduler's memory without bound (latent since MS2).* Every empty claim appended two history events and a receipt. We measured it in the harness container with four idle workers: the live heap grew **257 MiB per hour** (about 980 bytes per event), so a scheduler with its 128 MiB heap, left idle, would run out of memory in about half an hour. MS2's runs were too short to show it. An empty claim now records no event and replaces its session's previous idle receipt; per-session counters in `/v1/metrics` keep the polling evidence. After the fix the idle heap is flat. Logs: `results/ms3-logs/memory-probe/`. Real work still costs about 10 KB of heap per completed task, which bounds one run to about 11,000 tasks; the specification now states that capacity (§1).

C. Design choices and trade-offs

Leases instead of a failure detector. The scheduler cannot tell a slow worker from a dead one, so it does not try to. It only knows that it has not heard a renewal within the lease. Using leases means all timing lives on one clock (the scheduler's), so no clocks have to agree, and the existing pull model is kept: the scheduler still never contacts workers.

Attempt numbers as fencing tokens. A requeued task's next attempt has a higher attempt number. Every start, heartbeat and report names its attempt, and the scheduler accepts only the current, unexpired one. Outputs are written under an attempt-specific key, so a late worker cannot overwrite a newer attempt's files either. This fences everything the scheduler controls; it would not fence an external side effect such as a paid API call, which is out of scope.

Lease length 5 s, renewal every 1 s. Recovery time is roughly the lease, so a shorter lease recovers faster but expires live workers on slow links more often. With five renewals per lease, up to three consecutive heartbeats can always be lost without expiring a live worker; a fourth may cause expiry. A false expiry costs repeated work, never correctness, so we chose a lease that keeps false expiries rare on our hosts rather than the fastest recovery.

Expiry before every decision, plus a scan. If expiry happened only in a background scan, whether a late report is accepted would depend on scan timing. Checking deadlines at the start of every command makes the rule exact: after the deadline, the attempt is dead. The scan exists only so that an orphaned task is requeued even when no worker happens to claim.

A late success is rejected even if nobody took the task. A simpler alternative was to accept a success that arrives after the deadline when no other worker has claimed the task. We rejected it: one rule ("expired means dead") is easier to reason about and to test, and the cost is a re-run in a rare race.

Workers do not fence themselves. A worker that cannot reach the scheduler keeps working and lets the scheduler decide. Self-fencing would save some wasted work, but it would need the worker to estimate the scheduler's lease on its own clock, and safety does not need it.

Still one in-memory scheduler. Making the scheduler durable or replicated is a separate, large problem (a write-ahead log or consensus). The approved fault model is about workers, so we kept the scheduler out of scope and state plainly what a scheduler restart loses.

D. Testing

`make test` runs the whole suite in a pinned Linux container: 77 JUnit tests, then the pytest suites against real scheduler, artifact-store and worker processes. Every service-backed test exports the scheduler's full event history, worker logs and snapshots and runs the safety checker on them, so S1–S3 are checked in every test, not only in the ones aimed at them. The test-to-property table is in test-matrix.md.

Faults are injected with SIGKILL, SIGSTOP/SIGCONT, worker stall points, and a per-link proxy that logs every dropped, delayed or duplicated message (`proxy-worker-N.jsonl` in each test's evidence). Timing assertions use only the scheduler's clock, never differences between processes.

Key results from the submitted run (`make-test-3`):

- Leases expired 10 times across the fault tests, once for every injected crash, freeze, partition or heartbeat loss that outlasted the lease, and never in a fault-free test or in the slow-task and short-partition tests.
- Detection took 5.01–5.27 s from the last accepted renewal, never less than the 5 s lease.
- Recovery (last renewal → new owner) took 5.04–5.27 s when an idle worker was polling, and 6.04–6.05 s in the chaos run, where the recovered tasks waited behind queued work.

- In every one of those tests, the late worker's messages were rejected or abandoned, exactly one success per task was recorded, and the job finished with the correct output.

Do the tests test anything? For the lease logic we broke the code on purpose and checked that the tests failed. Stopping heartbeats from renewing the lease failed 2 lease tests. Skipping the expiry check before reports failed 2 others. The history checker's tests each inject one violation into a real saved history and require the checker to report it.

E. Performance

Method. `make bench-recovery` (`benchmarks/recovery.py`) uses the Step 12 workload and closed loop: six-task jobs with 100 ms fixture operations, 4 warmup jobs, then 24 measured jobs with at most C in flight. Here W = 4 workers and C ∈ {1, 4, 16}, with three fresh Compose deployments per configuration and mode (1 CPU and 512 MiB per service). *Baseline* injects nothing. *Crash* SIGKILLs a worker that is executing a task after the 6th and the 14th measured completions, and immediately starts a replacement worker. All times are on the scheduler's monotonic clock. Source `d4fcda9`; the service code is identical to the tested `966965e`. Raw data: `results/ms3-logs/recovery-benchmark/`.

C	Mode	Complete runs	Throughput, tasks/s (mean ± sd)	Job time p50 / p95 (ms)	Jobs hit by a kill: p50 (ms)	Recoveries	Recovery p50 / p95 / max (ms)	Detection p50 (ms)	Requeue → new owner p50 / max (ms)
1	baseline	3/3	4.40 ± 2.23	1,042 / 5,207	–	–	–	–	–
1	crash	2/3	4.03 ± 0.01	1,035 / 6,250	6,250	4	5,004 / 5,045 / 5,045	5,004	0 / 1
4	baseline	3/3	14.78 ± 1.33	1,563 / 1,805	–	–	–	–	–
4	crash	3/3	9.66 ± 0.24	1,690 / 6,656	6,872	5	5,058 / 5,103 / 5,103	5,053	0 / 64
16	baseline	3/3	11.85 ± 6.12	5,851 / 20,143	–	–	–	–	–
16	crash	3/3	9.92 ± 5.16	6,006 / 28,598	11,407	4	5,052 / 10,328 / 10,328	5,005	0 / 5,324

Percentiles are nearest-rank over the jobs of complete runs; throughput is the mean (standard deviation) of per-run values.

Recovery takes one lease. Each of the 16 kills was requested while its victim was executing a task. In 13 of them the task was still unfinished when the SIGKILL landed; on this host `docker kill` took up to about a second, and the other 3 victims had just reported. All 13 orphaned tasks were recovered:

- Detection took 5.00–5.09 s, never less than the 5 s lease.

- In 12 of 13, the task was reassigned within 64 ms of its expiry, for a recovery of 5.00–5.10 s.
- In one C=16 run every worker was busy, so the requeued task waited 5.3 s at the back of the queue, for a recovery of 10.3 s. That is what FIFO requeueing implies, and it is why P-REC is stated only for the case where an idle worker is polling.

What a crash costs a job. A job that lost a task to a crash took a median of 6.3 s (C=1), 6.9 s (C=4) and 11.4 s (C=16), against 1.0, 1.7 and 6.0 s for the other jobs in the same runs: roughly one lease plus a re-run of the task. Jobs that were not hit were not slowed; their medians were within 8% of the baseline's at every C (1.03 vs 1.04 s, 1.68 vs 1.56 s, 5.98 vs 5.85 s). With two crashes in a 24-job run, throughput fell from 14.8 to 9.7 tasks/s at C=4. At C=1 and C=16 the difference is within run-to-run noise.

Throughput and today's host. Fault-free throughput at W = 4 was 4.4, 14.8 and 11.9 tasks/s at C = 1, 4 and 16. Step 12 measured 6.0, 15.4 and 16.7 on the same laptop. The difference comes from the host, not from leases:

- **Variance today was large.** These runs began in a slow period: the first C=1 baseline ran at 1.8 tasks/s and the next two at 5.7.
- **A same-day control rules out the code.** The unchanged MS2 code at C=1, W=4 ran at 3.06 tasks/s, the same as MS3's code measured at the same time (2.99–3.30 tasks/s; `results/ms3-logs/perf-attribution/`).
- **Heartbeats add almost no traffic here.** They start one second into an attempt, and these tasks finish in a fraction of a second.

MS2's fault-free matrix (MS2 report §E) therefore remains our throughput characterization. An idle worker receives a ready task in about 5 ms; workers, not the scheduler, limit throughput.

Memory. After the MS3 fix, an idle scheduler's heap is flat. Real work costs about 10 KB of heap per completed task, so one run holds about 11,000 tasks with the default 128 MiB heap (`results/ms3-logs/memory-probe/`).

Completeness. 17 of the 18 runs completed. In the remaining one (crash, C=1, repetition 1), the benchmark driver's 6-second HTTP read of the event stream timed out during the slow period, and the driver stopped. The system had completed all 27 submitted jobs correctly, and both of that run's kills were recovered, but the run is excluded from the table. The histories of all 18 runs (503 jobs) pass the offline safety checker. Three earlier attempts were stopped and are kept as records (`results/ms3-logs/README.md`): one killed idle workers, one ran on code from before the memory fix, and one hit a driver race when starting replacement workers.

Environment. The same laptop as MS2: Windows 11, 12 CPUs, 16 GB RAM, Docker Desktop 27.1.1 with its Linux VM capped at 3 GB, shared with a browser and an IDE.

F. Deployment

`make up` starts the scheduler, the artifact store and three workers under Docker Compose (1 CPU and 512 MiB each, no automatic restarts), and `make demo` runs the functional and video workloads against them. The Hokea adapter for the course cluster from MS2 is unchanged except that it now packages

the fault proxy and expects zero expected failures. As in MS2, the course runner's Hokea version did not match the pinned one, so a cluster run is still unverified.

G. Limitations and known risks

- **The scheduler is a single point of failure.** Its state is in memory; a restart starts an empty run and in-flight jobs must be resubmitted.
- **One run holds about 11,000 completed tasks** with the default 128 MiB heap, because the full history is kept in memory for auditing (about 10 KB per task). Sustained full load reaches that in roughly 10–15 minutes; restart the scheduler (a new run) or give it a larger heap.
- **A paused scheduler expires live leases.** If the scheduler process itself is frozen for longer than the lease, every lease looks overdue when it resumes, and live workers' attempts are expired and redone. Safety holds; work is repeated.
- **At-least-once execution.** A task can run more than once: after a lease expiry, the old worker may have been frozen or cut off rather than dead and may finish too. Only one result is accepted. External side effects would need their own idempotency.
- **No retry limit.** An operation that always fails is retried forever and its job stays RUNNING.
- **Network faults are injected only on the worker↔scheduler link,** and only with the native backend. The artifact store is assumed available.
- **The artifact store's index is not rebuilt** after the store restarts, although its files stay on disk.
- **Workers are trusted;** results are not recomputed.
- **Performance numbers come from one shared Windows laptop** with a synthetic workload (§E).

H. Next steps (MS4–MS5)

Respond to the red team's findings and the plot twist. Candidates we already know about: an optional retry limit with a terminal FAILED job state, scheduler durability through a write-ahead log, and running the fault suites on the Compose and Hokea backends.